

Proxmox에서 cloud-init으로 클러스터 노드 만들기

VM 템플릿 제작부터 3노드 클론까지 — 2026년 8월 정리

i5-8400 / GTX 1060 3GB / DDR4 16GB짜리 유휴 PC에 Proxmox를 깔고, 그 위에 k8s 마스터 1대와 워커 2대를 올리는 게 목표였다. 처음엔 마스터를 Ubuntu ISO로 직접 설치했는데, 설치 마법사 중간에 비밀번호를 깜빡하는 바람에 다시 지우고, 이후로는 세 대 전부 cloud-init 템플릿 방식으로 통일했다. 여기 정리한 건 그 cloud-init 방식 기준이다.

Proxmox 설치 직후 손봐야 했던 것

설치가 끝나면 기본적으로 유료 저장소(enterprise)로 설정되어 있어서 `apt update` 가 401 에러로 막힌다. 이걸 무료 저장소로 바꿔야 패키지 설치가 된다.

```
cat > /etc/apt/sources.list.d/proxmox.sources << 'EOF'
Types: deb
URIs: http://download.proxmox.com/debian/pve
Suites: trixie
Components: pve-no-subscription
Signed-By: /usr/share/keyrings/proxmox-archive-keyring.gpg
EOF

apt update && apt dist-upgrade -y
```

기존 enterprise 저장소 파일(그리고 Ceph 저장소 파일도)은 이름 뒤에 `.disabled` 를 붙여서 비활성화해두면 된다. 단일 노드라 Ceph 자체가 필요 없어서 이쪽도 같이 꺼냈다.

스토리지는 이름이 비슷해서 헷갈렸다. `local` 은 ISO나 템플릿, 백업을 담는 디렉토리형 저장소고, `local-lvm` 은 VM/컨테이너 디스크 전용 thin pool이다. cloud-init 이미지를 임포트할 땐 `local-lvm` 쪽을 쓴다.

cloud-init 템플릿 만들기

1 Ubuntu cloud 이미지 다운로드

```
cd /var/tmp
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
```

설치 프로그램이 들어있는 ISO와 달리, cloud image는 이미 설치가 끝난 디스크 이미지다. 부팅하면 곧바로 cloud-init이 IP·계정·SSH키를 주입하고 끝난다. ISO는 매번 마법사를 손으로 눌러야 하지만, cloud image는 한 번 템플릿으로 만들어두면 `qm clone` 한 줄로 계속 복제할 수 있다는 게 차이다.

2 빈 VM 뼈대 생성

```
qm create 9000 \
  --name ubuntu2404-template \
  --memory 2048 \
  --cores 2 \
  --sockets 1 \
  --cpu host \
  --net0 virtio,bridge=vbr0 \
  --ostype l26
```

소켓은 물리 CPU 개수를 뜻한다. i5-8400은 1개짜리 CPU라서 소켓은 항상 1로 두고, 코어 수만 조정한다. 소켓을 2로 잡으면 존재하지 않는 CPU를 에뮬레이션하려다 오히려 손해다.

3 디스크 импорт + 연결

```
qm importdisk 9000 noble-server-cloudimg-amd64.img local-lvm
qm set 9000 --scsihw virtio-scsi-pci --scsi0 local-lvm:vm-9000-disk-0
```

4 cloud-init 드라이브 + 부팅/콘솔/에이전트 설정

```
qm set 9000 --ide2 local-lvm:cloudinit
qm set 9000 --boot order=scsi0
qm set 9000 --serial0 socket --vga serial0
qm set 9000 --agent enabled=1
```

세 줄이 각각 다른 일을 한다. `boot order` 는 여러 부팅 장치 중 실제 OS 디스크(scsi0)부터 부팅하도록 강제한다. `serial0 / vga serial0` 은 cloud image가 부팅 로그를 일반 VGA 화면이 아니라 시리얼 포트에 출력하기 때문에 필요한데, 이거 없이는 Proxmox 콘솔을 열어도 화면이 그냥 까맣게 보인다. `agent enabled=1` 은 QEMU Guest Agent용 채널을 열어서, 나중에 `qm guest cmd` 로 VM의 실제 IP를 조회하거나 정상 종료(graceful shutdown)를 시킬 수 있게 해준다.

5 디스크 확장 + 계정 설정

```
qm resize 9000 scsi0 20G

qm set 9000 --ciuser ubuntu
qm set 9000 --cipassword <비밀번호>
qm set 9000 --sshkey ~/.ssh/id_rsa.pub
```

cloud image 원본은 2~3GB로 작아서 미리 키워둔다. 계정과 SSH 공개키는 여기서 템플릿 레벨로 지정해두면, 이 템플릿에서 클론되는 모든 VM에 그대로 적용된다.

6 템플릿으로 전환

```
qm template 9000
```

이 순간부터 9000번은 직접 부팅할 수 없고, 클론 원본으로만 쓰인다.

템플릿에서 노드 3대 클론

```
qm clone 9000 101 --name k8s-master --full
qm clone 9000 102 --name k8s-worker1 --full
```

```
qm clone 9000 103 --name k8s-worker2 --full

qm set 101 --memory 3072 --cores 2 --ipconfig0 ip=192.168.1.101/24,gw=192.168.1.1
qm set 102 --memory 4096 --cores 2 --ipconfig0 ip=192.168.1.102/24,gw=192.168.1.1
qm set 103 --memory 4096 --cores 2 --ipconfig0 ip=192.168.1.103/24,gw=192.168.1.1

qm start 101
qm start 102
qm start 103
```

--full 로 완전히 복제한 이유는, 링크드 클론은 템플릿 디스크를 공유하기 때문에 템플릿을 지우면 클론된 VM도 같이 깨지기 때문이다. 마스터는 컨트롤플레인 전용이라 3GB, 워커는 실제 워크로드가 올라가는 곳이라 4GB로 나눠 잡았다. IP는 Proxmox 호스트 자체가 있는 192.168.1.0/24 대역에 맞춰야 서로 통신된다 — 처음에 이걸 192.168.0.x 로 착각해서 한 번 헤맸다.

개념 몇 가지

qm 은 QEMU 명령어가 아니라 Proxmox가 자체적으로 만든 관리 CLI다. 내부적으로 QEMU 프로세스를 직접 관리한다(libvirt는 안 쓴다). 지금 만드는 VM은 QEMU/KVM 기반이고, Proxmox의 LXC 컨테이너는 커널을 호스트와 공유하는 완전히 다른 기술이라 여기서는 다루지 않았다.

VM 만들 때 그래픽카드(Display) 설정은 물리 GPU와 아무 상관이 없다. 그냥 콘솔 화면을 보여주는 가상 어댑터일 뿐이다. GTX 1060을 VM이 실제로 쓰게 하려면 PCI Passthrough라는 완전히 별도의 작업(IOMMU 활성화, vfio-pci 바인딩, PCI 장치 추가)이 필요한데, 이건 나중에 GPU 워크로드를 올릴 때 따로 다룰 부분이라 지금은 기본값 그대로 두면 된다.